

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 7 — 2D Arrays

Why we need them

In Programming 6 an array was one row of boxes. Plenty of data comes as a **table** instead. Here are three students' marks in four tests.

	Test 1	Test 2	Test 3	Test 4
Ana	16	30	14	28
Ben	25	29	21	27
Chloe	9	12	8	11

A **2D array** stores the whole table under one name. Each row is one student, so it works like a database table: each row is a **record** and each column is a **field**.

VOCAB 2D array (OCR 2.2.3) — an array of arrays. Each row is itself an array, so the values sit in rows and columns like a table

Where you meet them. Whenever data comes in rows and columns, a program stores it as a 2D array.

Where	Each row is...	Each column is...	One box holds...
A spreadsheet	a row of cells	a column of cells	one cell, such as a mark
A game board	a row of squares	a column of squares	what is on that square: "X", "O" or empty
A bitmap image	a line of pixels	a pixel across	the colour code for that pixel
A train timetable	a station	a train	a departure time
A cinema seat plan	a row of seats	a seat number	True if booked
A database table	a record	a field	one item, such as a surname

One name and two numbers reach any box. Here is a game of noughts and crosses, stored as a 2D array.

```
board = [ ["X", "O", "X"],  
          [ "", "X", "O"],  
          ["O", "", "X"] ]
```

board[1][1] is the centre square, "X". The diagonal board[0][0], board[1][1], board[2][2] is X X X, so X has won. The program checks for a win by reading boxes, exactly as you will in Section 1.

With a loop inside a loop (Section 4), one short program visits every box. That is how photo software brightens every pixel in a picture, and how a markbook adds up every student's marks.

1 Make a 2D array and read one box

The rule. Write each row as an ordinary array in square brackets. Then put all the rows inside one more pair of square brackets, separated by commas. Putting each row on its own line keeps the table easy to read. Inside the brackets, Python ignores spaces and line breaks, so lining the rows up is for the reader: the array is the same either way.

```
marks = [[16, 30, 14, 28],
          [25, 29, 21, 27],
          [ 9, 12,  8, 11]]
```

Every box now has **two** index numbers: the **row** and the **column**. Both start at 0.

	col 0	col 1	col 2	col 3
row 0	16	30	14	28
row 1	25	29	21	27
row 2	9	12	8	11

To read a box, write the name, then the **row** in square brackets, then the **column** in square brackets. Row first, column second.

You type	Prints	What it does
<code>print(marks[0][0])</code>	16	Row 0, column 0: the top-left box.
<code>print(marks[1][2])</code>	21	Row 1, column 2: Ben's Test 3.
<code>print(marks[2][3])</code>	11	Row 2, column 3: the bottom-right box.
<code>print(marks[1])</code>	[25, 29, 21, 27]	One index gives a whole row, which is an ordinary array.
<code>print(len(marks))</code>	3	Counts the rows .
<code>print(len(marks[0]))</code>	4	Counts the boxes in row 0, which is the number of columns .

Now try • Type this array in. Each number tells you where it lives: 34 is in the row of thirties.

```
tens = [[ 1,  2,  3,  4,  5,  6],
         [11, 12, 13, 14, 15, 16],
         [21, 22, 23, 24, 25, 26],
         [31, 32, 33, 34, 35, 36]]
```

(a) Predict first. Write down what each line will print. Then run them.

```
print(tens[0][0])    print(tens[2][4])    print(tens[3][0])
print(tens[0][5])    print(len(tens))      print(len(tens[2]))
```

Check • Six lines: 1, 25, 31, 6, 4, 6.

(b) Write a line that prints each of these numbers from `tens`: 34, 6, 22, 31, 13.

Check • Each line prints the number you were aiming for.

2 Change one box

The rule. Name the box with its row and column on the left of =, and put the new value on the right. Every other box stays as it was.

```
tens[3][5] = 37
print(tens[3])
```

Line 1 reads as “row 3, column 5 becomes 37”: the 36 at the end of the bottom row becomes 37. Line 2 prints row 3 only: [31, 32, 33, 34, 35, 37].

Now try • Type the two lines above. Then change 14 to 17 and 4 to 7. Then print the whole array with `print(tens)`.

Check • [[1, 2, 3, 7, 5, 6], [11, 12, 13, 17, 15, 16], [21, 22, 23, 24, 25, 26], [31, 32, 33, 34, 35, 37]]. Python prints the rows one after another on a single line.

3 The user picks the box

A game does this on every turn: the player chooses a square, and the program changes that box.

The rule. The row and the column can be variables. Whatever numbers they hold decide which box is used.

Now try • Write a program that asks the user for a **row**, then a **column**, then a **new value**, and puts the new value in that box of `tens`. Finish by printing the row that changed.

Build it in four lines. Each `input()` needs `int()` around it, because the index numbers and the value are whole numbers. Put the `tens` line from Section 1 at the top, so it starts from the original numbers.

Check • Enter row 0, column 1, value 99. It prints [1, 99, 3, 4, 5, 6]: the 2 has become 99.

4 Visit every box with a loop inside a loop

First, a reminder: the for loop. A for loop runs its indented lines a set number of times. Each time round is one **lap**. The loop variable holds the lap number, starting at 0.

```
for i in range(3):
    print("Lap " + str(i))
```

`range(3)` makes the numbers 0, 1, 2, so this prints Lap 0, Lap 1, Lap 2. In Programming 6 you used exactly this to visit every box of a 1D array, with `i` as the index.

A loop inside a loop. The indented lines of a loop can be anything, including another loop. The **inner** loop then runs all the way through on **every** lap of the **outer** loop.

You read a page this way. For each line, you read every word from left to right, and only then move down to the next line. The lines are the outer loop; the words are the inner loop.

```
for row in range(3):
    for col in range(4):
        print(row, col)
```

A comma inside `print()` prints both values with a space between them.

Outer lap	The inner loop prints
row is 0	0 0 0 1 0 2 0 3
row is 1	1 0 1 1 1 2 1 3
row is 2	2 0 2 1 2 2 2 3

3 outer laps $\times$ **4 inner laps** = **12 lines**, each on its own line down the screen. To count the lines a nested loop prints, multiply the two lap counts.

VOCAB Nested loop (OCR 2.2.1) — a loop inside another loop. The inner loop runs all the way through on every lap of the outer loop

Now try • Change the code to `range(2)` on the outer loop and `range(5)` on the inner loop. **Predict** how many lines it will print, and what the first and last line will be. Then run it.

Check • 10 lines. The first is 0 0 and the last is 1 4.

Now look back at the grid in Section 1. Those pairs, 0 0 up to 2 3, are exactly the row and column of every box in a 3-row, 4-column array. So the nested loop visits every box, and `marks[row][col]` reads the box it is on.

The rule. To visit every box of a 2D array, use a nested loop. The **outer** loop counts through the rows. For each row, the **inner** loop counts through the columns.

```
for row in range(len(marks)):
    for col in range(len(marks[row])):
        print(marks[row][col])
```

Part of the code	What it does
<code>for row in range(len(marks)):</code>	<code>len(marks)</code> is 3, so row counts 0, 1, 2: one lap per row.
<code>for col in range(len(marks[row])):</code>	<code>len(marks[row])</code> is 4, so col counts 0, 1, 2, 3: one lap per column. It is indented once, so it runs inside each row.
<code>print(marks[row][col])</code>	Indented twice, so it runs once for every box.

Here is the order it visits the boxes. The inner loop finishes a whole row before the outer loop moves down.

row	col	marks[row][col]	Prints
0	0	marks[0][0]	16
0	1	marks[0][1]	30
0	2	marks[0][2]	14
0	3	marks[0][3]	28
1	0	marks[1][0]	25
... and so on, to row 2, column 3 ...			

Three rows of four boxes: **12** lines come out. Photo software works the same way: to brighten a picture, it visits every pixel with a nested loop and changes each one.

Now try • (a) Use a nested loop to print every number in tens. Use the original tens from Section 1.

Check • 24 lines, starting at 1 and ending at 36.

(b) Add up **every** mark in marks. Start total at 0 above both loops, add each box to it in the inner loop, and print it once at the very end, with no indent.

Check • 230.

5 One total for each row

A markbook needs one total for each student, not one for the whole class.

The rule. To total each row separately, set total back to 0 at the start of every row, and print it at the end of every row. Both of those lines belong to the **outer** loop, so they are indented once.

```
for row in range(len(marks)):
    total = 0
    for col in range(len(marks[row])):
        total = total + marks[row][col]
    print("Student " + str(row) + " total: " + str(total))
```

Line	Indent	When it runs
total = 0	once	At the start of each row, so every student starts from zero.
total = total + ...	twice	For every box in that row.
print(...)	once	After the inner loop has finished the row.

It prints three lines: Student 0 total: 88, Student 1 total: 102, Student 2 total: 40. The str() is the casting from Programming 3, because a number is being joined to text.

Now try • Print the total of each row of tens (the original from Section 1).

Check • Four lines: 21, 81, 141, 201.

6 Fill a 2D array from the keyboard

This is how a markbook or a seat plan gets filled in the first place: one box at a time.

The rule. First make the table the right size with every box set to 0. Then use a nested loop to put an input into each box.

```
scores = [[0, 0, 0],
          [0, 0, 0]]
for row in range(len(scores)):
    for col in range(len(scores[row])):
        prompt = "Student " + str(row) + ", test " + str(col) + ": "
        scores[row][col] = int(input(prompt))
print(scores)
```

Lines 1–2 make 2 rows of 3 boxes: two students, three tests. Line 5 builds the message from row and col, so the user sees Student 0, test 1: and knows which box they are filling. Line 6 asks with that message and stores the answer.

Now try • Type the program and run it. Enter 7, 9, 5, then 10, 6, 8.

Check • [[7, 9, 5], [10, 6, 8]]. The first three went into row 0, in order.

Now add the row totals from Section 5 underneath.

Check • Student 0 total: 21 and Student 1 total: 24.

Stretch The user chooses the size

Write a program that:

1. asks **how many students** and **how many tests**
2. makes a 2D array exactly that size
3. asks for every mark and stores it
4. prints each student's **average**.

Build it one line at a time, running after each. Line 2 is the only new idea. This line makes students rows, each holding tests zeros:

```
marks = [[0] * tests for r in range(students)]
```

Check • Choose 2 students and 3 tests, and enter 7, 9, 5, 10, 6, 8. The averages are 7.0 and 8.0.

The exam writes it differently

In OCR's exam language, ERL, both index numbers go inside **one** pair of square brackets, separated by a comma. Python gives each one its own brackets.

ERL (what you read)	Python (what you type)	Meaning
array marks[3,4]	marks = [[0]*4 for r in range(3)]	3 rows of 4
print(marks[1,2])	print(marks[1][2])	Row 1, column 2
tens[3,5] = 37	tens[3][5] = 37	Change one box
marks.length	len(marks)	How many rows
marks[0].length	len(marks[0])	How many columns

The nested loop in ERL. Each loop closes with its own next, inner first.

```
for row = 0 to marks.length - 1
    for col = 0 to marks[row].length - 1
        print(marks[row,col])
    next col
next row
```

As with 1D arrays, ERL names the last index, so it writes - 1. Python's range() stops one short on its own. Both visit every box.